

TECHNICAL DOCUMENTATION

JotFlow

Workflow Approval Platform · End-to-End System Workflow

Client: Government of Dubai Media Office (GDMO)

Engine: JotForm Enterprise Workflow API

Stack: React + TypeScript + Vite | Express + PostgreSQL | Windows IIS

Document version: 1.0 · Generated 2026-06-27

Contents

1. Executive Overview
2. System Architecture
3. Authentication & Access Gate
4. Data Ingestion & Synchronization
5. The Approval Workflow Lifecycle
6. The Three Workflow Task Types
7. Visibility & Access Control
8. Frontend Data Flow
9. Key Invariants & Design Rules
10. Glossary

1. Executive Overview

JotFlow is a workflow approval dashboard that wraps the JotForm Enterprise workflow engine for the Government of Dubai Media Office. It gives each employee a single personal queue of the approvals, forms, and tasks awaiting their action, and lets them act on those items (approve, reject, sign, fill a form) without leaving the dashboard.

The platform sits **on top of** JotForm Enterprise rather than replacing it. JotForm remains the source of truth for form definitions and workflow instances. JotFlow mirrors that data into a local PostgreSQL database so the dashboard can render fast, filter by participation, and stay live over WebSockets, while every state-changing action is pushed back to JotForm's workflow API.

The central design principle: dual-pass authority.

The first pass reads plain form fields (legacy, kept for backward compatibility). The second pass reads the JotForm *workflow instance* (its status plus the task list). The second pass is always authoritative. Whenever the two disagree, the workflow instance and its task list win, and stale form-status columns are ignored.

What the platform does, in one paragraph

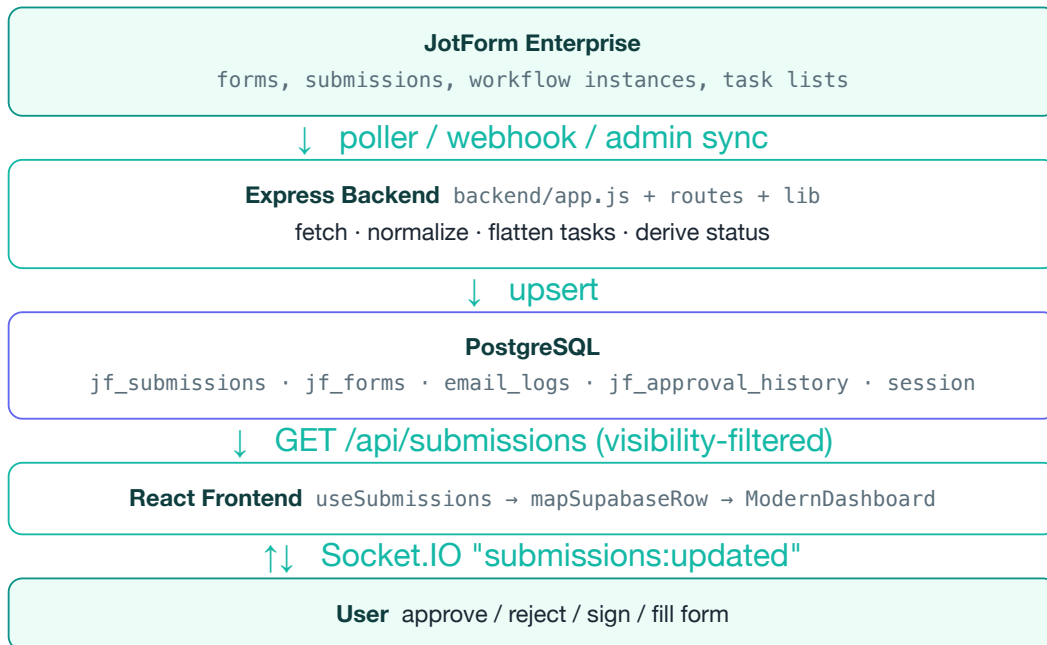
JotForm emits workflow data (via a background poller, a push webhook, or an admin bulk sync). JotFlow normalizes that data into the `jf_submissions` table, flattening each workflow task so the assignee email is available at the top level. The frontend reads only rows the logged-in user participates in, maps each row into a typed `Submission` object, and renders a personal action queue. When a user approves or rejects, the action is sent to `POST /workflow/task/{taskId}/complete` on JotForm; the result drives notifications and a real-time refresh.

Core building blocks

Layer	Technology	Responsibility
Frontend	React 18, TypeScript, Vite 5, Tailwind, Framer Motion, Socket.IO client	Personal action queue, approve/sign UI, live updates
Backend	Express 4, Socket.IO, Helmet, express-rate-limit, Pino, Zod	API, sync, workflow actions, auth, visibility enforcement
Database	Self-hosted PostgreSQL (<code>pg</code>), <code>connect-pg-simple</code> sessions	Mirror of JotForm data, sessions, approval history
External engine	JotForm Enterprise API + Workflow Engine API	Source of truth for forms and workflow instances
Deploy target	Windows IIS via iisnode (frontend optionally on Vercel)	Reverse proxy to the Express server on localhost:3001

2. System Architecture

JotFlow is a three-tier application with an external dependency on JotForm. The Express backend is the only component that talks to JotForm directly; the React frontend never calls JotForm itself, it reads the mirrored data from the backend.



Backend startup and middleware

The process entry is `backend/server.js`, which starts the HTTP server plus the Socket.IO server and installs a SIGTERM handler for graceful shutdown. Under IIS, a thin `server.js` at the repository root wraps this via `iisnode`, and `web.config` reverse-proxies requests to `localhost:3001`.

The Express app itself is built by the factory in `backend/app.js`, which assembles the middleware chain in order:

1. **Helmet** for security headers (CSP, HSTS, X-Frame-Options).
2. **Trust proxy** so client IPs survive the IIS ARR relay.
3. **CORS** (production defaults to the Vercel frontend origin; `ALLOWED_ORIGIN=*` for local dev).
4. **Rate limiting** with a global limiter plus tighter per-endpoint limiters (auth, API).
5. **Session** middleware backed by PostgreSQL through `connect-pg-simple`.
6. **Routes** for auth, submissions, workflow actions, webhooks, and admin sync.
7. **Error handler** as the final stage.

Health probes: `/api/health` is a shallow liveness check; `/api/health?deep=1` and `/api/health/ready` probe PostgreSQL, suitable for IIS / load-balancer readiness checks.

The key-type abstraction

Every JotForm call is keyType-aware. `backend/lib/key-type.js` exposes `readKeyType(req)`, which defaults to `'gdm0'` (the only configured key). `backend/lib/jotform.js` wraps `fetch` with an `AbortController` timeout, the `APIKEY` header, and rate-limit-aware retry. The GDMO key is scoped to enterprise tenant paths, so requests are routed to `/enterprise/*` endpoints rather than the generic `/user/*` endpoints.

3. Authentication & Access Gate

JotFlow supports two login methods and layers a workspace-membership gate on top of both, so that only people who actually participate in GDMO workflows can hold an account.

Method	Route	Mechanism
Local password	<code>backend/routes/auth-local.js</code>	bcrypt hash (12 salt rounds) against <code>users.password_hash</code>
Microsoft / Azure AD SSO	<code>backend/routes/auth-microsoft.js</code>	<code>@azure/msal-node</code> OAuth, email matched to a workspace member

The workspace membership gate

Implemented in `backend/lib/workspace.js`, the gate decides whether an email belongs to the workspace by checking whether it appears anywhere in the mirrored data: as a submitter, a pending approver, an assignee in `email_logs`, or an assignee inside a `workflow_tasks` entry. In development the gate is bypassed (always a member); in production a non-member is rejected with `403 not_workspace_member`.

Signup sequence

1. Client posts `{ email, password, fullName, department }`.
2. Workspace gate runs; non-members are rejected (403).
3. Duplicate email check (409 on conflict).
4. Password hashed with bcrypt and a `users` row is created.
5. A `profiles` row (role `viewer`) and an `org_members` row are created.
6. Session fields are set (`userId`, `email`, `role`, `fullName`).

Login sequence

1. Look up the user and profile (LEFT JOIN).
2. Verify the bcrypt hash.
3. Run the server-side workspace gate again.
4. Fire a background `pollOnce()` so the dashboard data is fresh on first paint (guarded by a single-run semaphore).
5. Set the session and return the profile, including `orgId`.

Sessions are stored in a PostgreSQL `session` table via `connect-pg-simple`, with a 30-minute idle timeout, `SameSite=Lax` cookies for CSRF defense, and the `Secure` flag in production.

4. Data Ingestion & Synchronization

There are three independent paths that bring JotForm data into PostgreSQL. All three apply the same normalization rules, so the resulting rows are identical regardless of which path wrote them.

Path	File	Trigger	Scope
Background poller	backend/lib/poller.js	ENABLE_POLLER=1 on an interval	Enabled forms, recent submissions
Webhook (push)	backend/routes/submissions-jotform.js	JotForm POST on change	A single submission
Admin bulk sync	backend/routes/admin-sync.js	Settings → "Sync All" (SSE)	All forms, all submissions

Path A: Background poller

The poller fetches all enabled forms, upserts their metadata to `jf_forms`, and for each form pulls submissions with `addWorkflowStatus=1`. For any submission that has a `workflowInstanceID` and is still active, it performs a **second pass**: it fetches the workflow instance, flattens each task with `extractTask()`, resolves prefill links for form-filler tasks, and derives the authoritative status. Enrichment runs with bounded concurrency (four at a time) and an inter-form delay to stay within JotForm rate limits. When the pass finishes it emits the Socket.IO event `submissions:updated`.

Path B: Webhook

The webhook endpoint verifies a shared secret, then fetches the one changed submission, runs the same flatten-and-derive logic, upserts the row, refreshes `email_logs` and task tokens, and emits the same Socket.IO event. This gives near-real-time updates between poll cycles.

Path C: Admin bulk sync

Triggered from Settings, the bulk sync streams progress over Server-Sent Events. It fetches every form and paginates through every submission (with a hard page cap and a truncation warning), enriches tasks with bounded concurrency, and reports statistics: forms synced, submissions upserted, and any truncation warnings.

Flatten invariant. Every writer (poller, webhook, admin sync) must flatten tasks through `extractTask` so that `assigneeEmail` lives at the top level of each `workflow_tasks` entry. If a writer stores the raw JotForm task list instead, assigned-user visibility silently breaks. The upsert also merges `workflow_tasks` as JSONB so an existing access link is preserved when an incoming payload omits it.

The `jf_submissions` row shape (selected columns)

```
jotform_submission_id      (primary key)
form_id, form_title, title, description
submitter_email, department, submission_date
status                    'pending' | 'approved' | 'rejected' | 'completed'
current_level, priority, amount
approver_name, approver_email
pending_approver_name, pending_approver_email
```

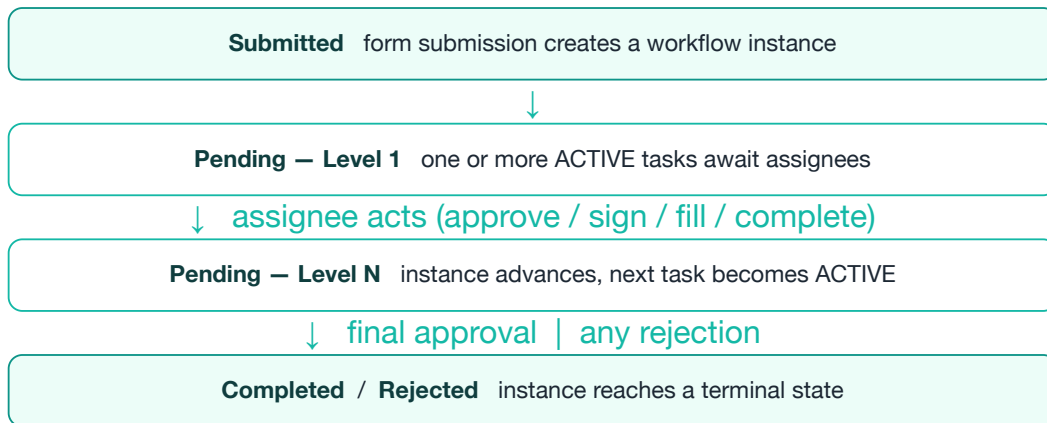
answers	JSONB	all form answers
workflow_tasks	JSONB	flattened task array (assigneeEmail at top level)
level_history	JSONB	[{ level, status, approver, approverEmail, date }]
raw_data	JSONB	original JotForm response
last_synced, created_at_jf, updated_at_jf, approval_url		

5. The Approval Workflow Lifecycle

A workflow moves through states that are computed from the JotForm workflow instance, not from form dropdowns. The function `deriveWorkflowStatus()` in `backend/lib/workflow-task.js` is the single place that resolves a row's status.

Status derivation rules (in order)

1. If the instance status is `COMPLETED` / `COMPLETE` → **completed**.
2. If the instance status is `REJECTED` / `CANCELLED` / `DECLINED` → **rejected**.
3. If any task is `ACTIVE` or `PENDING` → **pending**.
4. If an end-point task is completed, or every task is completed → **completed**.
5. If the instance is `ACTIVE` / `PENDING` with no tasks → **pending**.
6. Otherwise return `null` : no signal, so keep the existing stored status.



The approval action: `POST /api/workflow-action`

Implemented in `backend/routes/submissions-actions.js`, this endpoint turns a user's click into a JotForm task completion.

1. Client sends `{ submissionId, taskId, action, comment, signature }`.
2. The backend resolves the `workflowInstanceID` from the DB or JotForm and fetches the live task list.
3. It locates the `ACTIVE` task assigned to the logged-in user (by `taskId` if given, otherwise by email match, with a DB fallback).
4. It reads the task's outcomes and resolves an `outcomeID`:
 - `approve` → the outcome of type `APPROVE` (or the first outcome).
 - `reject` → the outcome of type `DENY` (fails with 400 if none exists).
 - `complete` → the first outcome.
5. It calls JotForm: `POST /workflow/task/{taskId}/complete` with `{ outcomeID, comment, signature }`.
6. On success it records to `jf_approval_history`, sends notifications (to the submitter, and to the next approver if the instance is not yet complete), and emits `submissions:updated`.

JotForm's response includes `content.instanceCompleted`. When true, the workflow is finished and the pending-approver fields are cleared; otherwise the next approver has been assigned and the poller will enrich

the new state on its next cycle.

6. The Three Workflow Task Types

The JotForm workflow engine defines three kinds of active step. JotFlow detects the type of the user's ACTIVE task and renders a matching call-to-action. The mapping lives in `getMyActionType()` in `frontend/src/config/currentUser.ts`.

Task type	What the user does	CTA button	UI surface
approval <code>workflow_approval</code>	Review a pre-filled form, approve or reject, optionally sign	Review & Approve	SubmissionModal (in-app sign flow)
form <code>workflow_assign_form</code>	Fill out a different, pre-populated form	Fill Form	WorkflowDetailsSidebar → external JotForm form
task <code>workflow_assign_task</code>	Perform and confirm an external action	Open Task	WorkflowDetailsSidebar → external task link

Type 1: `workflow_approval`

The approver sees the submission and chooses approve or reject; a signature may be captured and stored in `workflow_tasks[].signatureUrl`. The action goes through the in-app `SubmissionModal` and resolves to the JotForm `APPROVE` or `DENY` outcome. The modal auto-closes on success.

Type 2: `workflow_assign_form`

The assignee must complete a separate form. JotFlow opens the JotForm form through a per-user prefill link resolved during enrichment. This is the case behind the recent fix that ensured the link resolves to the direct `/prefill/` URL rather than a `/share/` URL.

Type 3: `workflow_assign_task`

The assignee performs an external action (a verification, a BPM step). The task is opened through its access link and completed externally or through the workflow action with `action: 'complete'`; the poller then picks up the status change.

Parallel approvals. Multiple tasks can be ACTIVE at the same level at once. The single `pending_approver_email` column names only one of them, so the task list is the authoritative source. This is why `isAwaitingMyAction()` checks `workflowTasks[]` for an ACTIVE entry matching the user, and only falls back to the pending-approver field when no task list has synced yet.

```
// getMyActionType() in frontend/src/config/currentUser.ts
const active = submission.workflowTasks?.find(
  t => t.status === 'ACTIVE' &&
    t.assigneeEmail?.toLowerCase() === userEmail?.toLowerCase()
);
if (!active) return null;
switch (active.type) {
  case 'workflow_approval': return 'approval'; // Review & Approve
  case 'workflow_assign_form': return 'form'; // Fill Form
  case 'workflow_assign_task': return 'task'; // Open Task
}
```

7. Visibility & Access Control

Access is participation-based and default-deny. Only a `super_admin` sees every workflow. Every other user sees a row only if they are involved in it. The rule is enforced on the server before any data leaves the backend, and mirrored on the client for UI filtering.

Server-side gate

`backend/lib/visibility.js` is the single source of truth. `isRowVisible(row, userEmail, role)` returns true for admins, otherwise true only if the user's email is in the row's participant set computed by `rowParticipantEmails()`. That set is gathered from the submitter, the pending and historical approvers, every `level_history` entry (including emails parsed out of action text), and every `workflow_tasks` assignee or completer. `filterVisibleRows()` applies this to list endpoints.

Source field	Why it grants visibility
<code>submitter_email</code>	The user created the request
<code>pending_approver_email</code>	The user currently owes an action
<code>approver_email / level_history[].approverEmail</code>	The user approved at some level
<code>workflow_tasks[].assigneeEmail</code>	The user is assigned a task (covers parallel steps)
<code>workflow_tasks[].submittedByEmail</code>	The user completed a task

Client-side mirror

`frontend/src/config/currentUser.ts` provides `isSubmissionVisible()`, `isAwaitingMyAction()`, `getMyActionType()`, and `getMyWorkflowRole()`. These never widen access beyond the server; they decide which of the already-permitted rows belong on the personal queue versus the Completed and Pending-With tabs.

No admin bypass on the dashboard. The Modern Dashboard's personal action queue applies the same `isAwaitingMyAction()` rule to every user, so even an admin sees only their own pending items there. Admin breadth applies to the oversight tabs, not the personal queue.

8. Frontend Data Flow

The dashboard is database-first: it reads the mirrored rows from the backend, maps them into typed objects, caches them, and stays live over Socket.IO.

Load sequence (useSubmissions.ts)

1. On mount, call `loadFromSupabase({ force: true })`.
2. Fetch the active form scope from `GET /api/active-form-ids`.
3. Read rows from `GET /api/submissions?form_ids=...&limit=20000&order=desc` (already visibility-filtered server-side).
4. Map each row with `mapSupabaseRow()` into a typed `Submission`.
5. Cache to `localStorage` under a key scoped by JotForm key type, with a 30-minute TTL, for instant paint on reload.
6. Subscribe to Socket.IO `submissions:updated`; on each event, reload.
7. Diff new versus previous rows to raise toasts for new arrivals and status changes.

Row mapping and approver resolution

`mapSupabaseRow()` in `frontend/src/hooks/submissionMappers.ts` parses the JSONB columns and builds the approval history. Approver identity is resolved through `enrichApproverInfo()` using a priority chain that differs by state:

For acted levels (approved / rejected)	For pending levels
1. Parsed action text → 2. approver config table → 3. workflow task data → 4. form field → 5. "Level N Approver"	1. Workflow task data → 2. form field → 3. "Level N Approver" (config is intentionally not used for pending)

Live updates

Whenever the poller, the webhook, or a workflow action changes data, the backend (`backend/lib/realtime.js`) emits `submissions:updated` to all connected clients, which triggers a refresh. The dashboard therefore reflects approvals made elsewhere within seconds, without a manual reload.

9. Key Invariants & Design Rules

1. **Workflow instance is authoritative.** Status comes from the instance and its task list, never from form-status dropdowns. `deriveWorkflowStatus()` is the only resolver.
2. **Tasks must be flattened.** Every writer flattens through `extractTask` so `assigneeEmail` is top-level; otherwise assigned-user visibility breaks.
3. **Visibility is server-enforced and default-deny.** Only `super_admin` sees all; the backend filters rows before responding, and the client only narrows further.
4. **Parallel approvals require task-list logic.** The single pending-approver column cannot represent several simultaneous approvers; check `workflowTasks[]` `ACTIVE` status.
5. **Every JotForm call is keyType-aware.** Default to `'gdmo'`; the enterprise key uses `/enterprise/*` tenant paths.
6. **The CTA matches the active task type.** approval → in-app sign modal; form and task → external JotForm link.
7. **Build after changes.** Run `npm run build` to verify TypeScript and the bundle before shipping.

Mental model. JotFlow is a fast, participation-filtered, real-time read model over JotForm Enterprise, plus a thin write path that forwards approvals to JotForm's workflow API. Treat JotForm as the system of record and the local database as a derived cache that must always be reconcilable from the workflow instance.

10. Glossary

Term	Meaning
Workflow instance	A running JotForm workflow for one submission, with a status and a task list. Authoritative source of truth.
Task	One step in a workflow instance, with a type, a status (ACTIVE / PENDING / COMPLETED), and an assignee.
Flatten	Lifting <code>assigneeEmail</code> and related fields to the top level of a task via <code>extractTask</code> .
Second pass	The enrichment step that fetches the workflow instance and derives authoritative status.
Personal action queue	The Modern Dashboard view of items awaiting the logged-in user, filtered by <code>isAwaitingMyAction()</code> .
keyType	Selector for which JotForm API key and path scope to use; defaults to <code>gdmo</code> .
Outcome	A JotForm approval result option (<code>APPROVE</code> / <code>DENY</code>) identified by an <code>outcomeID</code> .

JotFlow technical workflow documentation. Generated from the codebase on 2026-06-27. File references point at the repository at the time of generation and may shift as the code evolves.